

Pattern Recognition and Neural Networks

Assignment 3

Bitupan Arandhara SR No: 04-03-00-10-22-25-1-25910

Phase 1: Tree Ensembles & Boosting

1.1 Information Gain from Scratch

The Delhi AQI dataset does not contain a “Temperature” column so the feature “no2” is used as a surrogate. The Gini impurity of a node with class distribution $\{p_k\}$ is defined as

$$G = 1 - \sum_k p_k^2.$$

For a binary split at threshold t on feature x_j , the Gini gain is

$$\Delta G(t) = G_{\text{parent}} - \left(\frac{|L|}{n} G_L(t) + \frac{|R|}{n} G_R(t) \right),$$

where $L = \{i : x_{ij} \leq t\}$ and $R = \{i : x_{ij} \geq t\}$. The model iterates over all unique midpoint thresholds and picks

$$t^* = \arg \max_t \Delta G(t).$$

Feature	Optimal Threshold	Gini Gain
no2	58.6050	0.143767

1.2 Random Forest

A Random Forest was implemented with the following:

- **Bootstrap sampling:** Each of the $T = 20$ trees is trained on a distinct bootstrap sample drawn with replacement from the training set.
- **Feature bagging:** At every node split, only $\lfloor \sqrt{p} \rfloor$ features are randomly selected as candidates, where p is the total number of features.
- **Depth limiting:** Trees are grown to a maximum depth of 5.
- **Prediction:** Majority vote across all trees.

The ensemble prediction is $\hat{y} = \text{mode}(\{h_t(x)\}_{t=1}^T)$, where h_t is the t -th tree.

Metric	Validation	Test
Accuracy	96.34%	97.37%

1.3 AdaBoost Sample Reweighting

At each boosting round t :

Step 1 – Train weak learner h_t on the weighted distribution $\{w_i^{(t)}\}$.

Step 2 – Compute weighted error:

$$\varepsilon_t = \frac{\sum_i w_i^{(t)} \mathbf{1}[h_t(x_i) \neq y_i]}{\sum_i w_i^{(t)}}.$$

Step 3 – Compute learner confidence:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right).$$

Step 4 – Update sample weights:

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t y_i h_t(x_i)), \quad \text{then normalise.}$$

For a consistently misclassified sample where $y_i h_t(x_i) = -1$ at every round, the weight grows as $\prod_t e^{\alpha_t}$, which is *exponential* in the number of iterations.

The 5 most consistently misclassified training samples were identified, and their weights were tracked over 50 boosting iterations.

Metric	Validation	Test
Accuracy	96.24%	96.84%

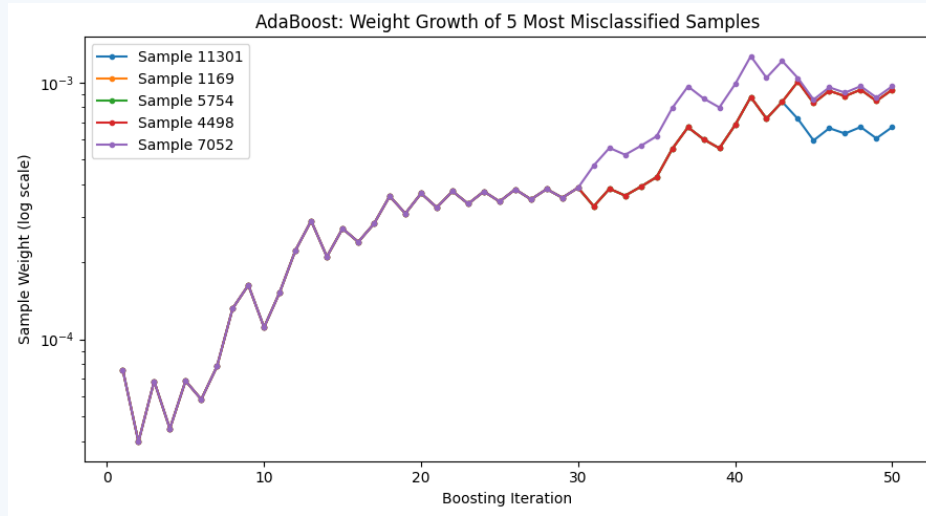


Figure 1: Exponential weight growth (log scale) for the 5 most misclassified training samples over 50 AdaBoost iterations.

1.4 Gradient Boosting Residuals (Regression)

Gradient Boosting for regression minimises the MSE loss $\mathcal{L} = \frac{1}{2}(y - F(x))^2$. The negative gradient (pseudo-residual) at each step is

$$r_i^{(t)} = -\frac{\partial \mathcal{L}}{\partial F(x_i)} = y_i - F^{(t-1)}(x_i).$$

A shallow regression tree f_t is fit to these residuals, and the model is updated as

$$F^{(t)}(x) = F^{(t-1)}(x) + \eta \cdot f_t(x),$$

with learning rate $\eta = 0.1$ and $F^{(0)} = \bar{y}$ (global mean).

Boosting Stage	1	5	10	50
Residual Variance	44 255.23	20 105.11	7 874.06	497.03

The test MSE on the continuous PM_{2.5} target is **370.90**.

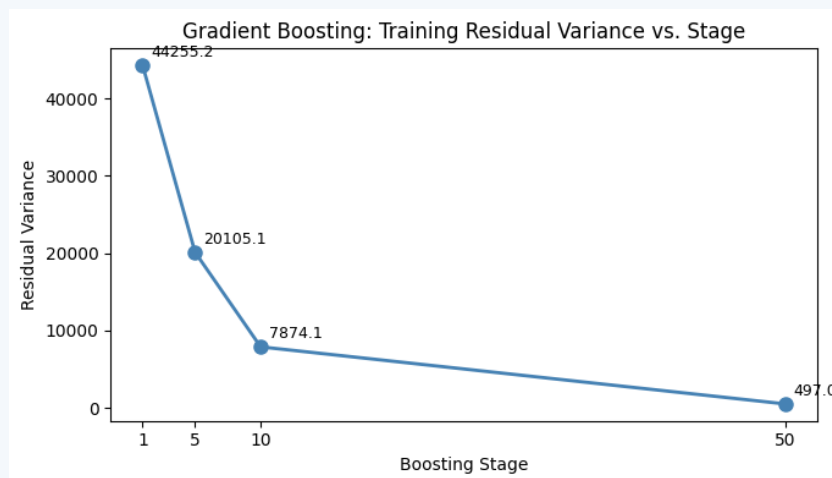


Figure 2: Training residual variance vs. boosting stage for Gradient Boosting.

Phase 2: Subspace & Clustering

2.5 PCA vs. Linear Autoencoders

PCA via SVD. The centred data matrix $X_c \in \mathbb{R}^{N \times 504}$ is decomposed as $X_c = U\Sigma V^\top$. The minimum components retaining at least 95% of the variance is $k = 75$.

A purely linear AE is trained with bottleneck dimension $k = 75$.

Method	Test Reconstruction MSE
PCA (SVD, $k = 75$)	0.052612
Linear AE ($k = 75$)	0.053280
Difference	0.000669

2.6 GMM Latent Clustering

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]}.$$

Metric	Value
GMM components	38
Embedding dimension	256
Adjusted Rand Index (ARI)	0.3256

An ARI of 0.33 indicates that the CNN's learned representations carry a meaningful class structure, even though the clustering is far from perfect.

Phase 3: Deep Generative Models

3.7 ELBO Implementation

Architecture. The encoder uses four strided conv layers ($3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ channels, stride-2) followed by two parallel linear heads for μ and $\log \sigma^2$. The decoder mirrors this with transposed convolutions ending in a Sigmoid output.

ELBO loss. The evidence lower bound decomposes as:

$$\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{Reconstruction (BCE)}} - \underbrace{D_{\text{KL}}(q_\phi(z|x) \parallel p(z))}_{\text{Regularisation}}.$$

With a diagonal Gaussian posterior $q_\phi(z|x) = \mathcal{N}(\mu, \text{diag}(e^{\log \sigma^2}))$ and standard normal prior $p(z) = \mathcal{N}(0, I)$, the KL term has a closed form:

$$D_{\text{KL}} = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The reparameterisation trick $z = \mu + \varepsilon \odot \sigma$, $\varepsilon \sim \mathcal{N}(0, I)$ makes the sampling step differentiable.

The reconstruction loss decreases steadily while the KL term *increases*, which is the expected: as the posterior becomes less like the prior, the KL penalty grows, but the reconstruction improves because more information is encoded in z .

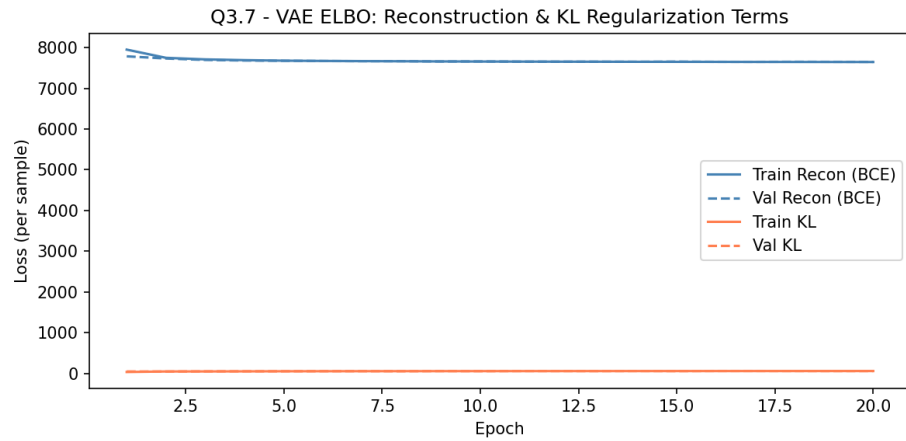


Figure 3: VAE ELBO components over 20 training epochs.

3.8 VAE vs. Simple DCGAN

DCGAN Architecture.

Generator ($z \in \mathbb{R}^{100} \rightarrow \mathbb{R}^{3 \times 64 \times 64}$): Five transposed conv layers ($100 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 3$) with BatchNorm and ReLU activations, ending in Tanh.

Discriminator ($\mathbb{R}^{3 \times 64 \times 64} \rightarrow \mathbb{R}$): Five conv layers using LeakyReLU (slope 0.2) and BatchNorm (except first layer), ending in a scalar logit.

Training objective:

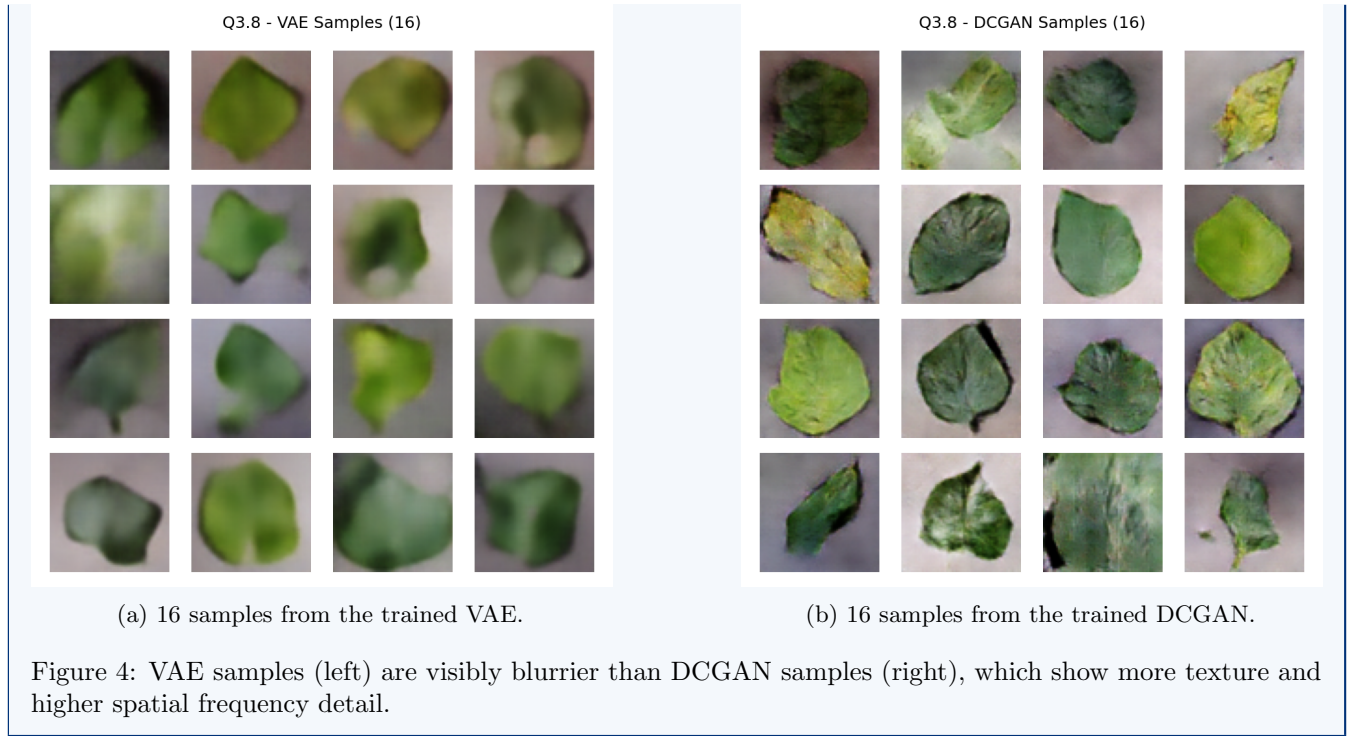
$$\min_G \max_D \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))].$$

In practice, the non-saturating variant is used for the generator: $\max_G \mathbb{E}_z[\log D(G(z))]$.

Epoch	Generator Loss	Discriminator Loss
5	3.6129	0.5228
10	4.2917	0.3373
20	4.8982	0.2047

The discriminator loss falling while the generator loss rises is a typical DCGAN training pattern: the discriminator becomes increasingly confident, forcing the generator to work harder.

Why VAE produces blurrier images. The VAE ELBO reconstruction term is a pixel-wise BCE (or MSE), which optimises $\log p_\theta(x|z)$ by minimising per-pixel differences averaged over the approximate posterior. For any ambiguous region of the image (where multiple plausible pixel values exist), the decoder minimises expected pixel error by outputting the *mean* of possible values - which is inherently blurry. The GAN generator faces no such averaging: it only needs to produce a single sharp image that fools the discriminator. Since D is trained on sharp real images, blurry outputs are immediately penalised, creating adversarial pressure toward crispness.



3.9 Fréchet Inception Distance (FID)

Setup. The APTOS dataset was filtered to Class 4 (proliferative diabetic retinopathy), yielding 295 training images. A fresh DCGAN was trained on this subset for 50 epochs. FID was then computed between 500 real and 500 generated images.

Feature extraction. Both sets of images were passed through a frozen pre-trained `inception_v3` (with `.fc` replaced by `nn.Identity()`) to extract 2048-dimensional pool features. The real images were resized to 299×299 directly; the generated 64×64 images were upsampled to 299×299 before passing through Inception.

FID formula. Given feature statistics (μ_r, C_r) for real and (μ_f, C_f) for fake:

$$\text{FID} = \|\mu_r - \mu_f\|^2 + \text{Tr}(C_r + C_f - 2\sqrt{C_r C_f}).$$

The matrix square root $\sqrt{C_r C_f}$ is computed via `scipy.linalg.sqrtm`. Small imaginary components from numerical precision are discarded.

```
from scipy.linalg import sqrtm
def compute_fid(feats_real, feats_fake):
    mu_r, mu_f = feats_real.mean(0), feats_fake.mean(0)
    C_r = np.cov(feats_real, rowvar=False)
    C_f = np.cov(feats_fake, rowvar=False)
    mean_sq_diff = np.sum((mu_r - mu_f) ** 2)
    sqrt_Cr_Cf = sqrtm(C_r @ C_f)
    if np.iscomplexobj(sqrt_Cr_Cf):
        sqrt_Cr_Cf = sqrt_Cr_Cf.real
    return float(mean_sq_diff + np.trace(C_r + C_f - 2 * sqrt_Cr_Cf))
```

Metric	Value
Class-4 training images	295
FID images per set	500
Feature dimension	2048
FID Score	255.94

The FID of 255.94 is relatively high, which is understandable given the very limited training set (only 295 Class-4 images). With so few samples, the generator struggles to capture the full distribution of retinal fundus images. Typical high-quality GANs achieve $\text{FID} < 30$ on larger datasets.

3.10 Batch Normalisation & Mode Collapse

Experimental setup. All BatchNorm layers were removed from the APTOS Generator (Discriminator unchanged). The generator uses `bias=True` instead. It was retrained from scratch on the same Class-4 APTOS

data using identical hyperparameters (50 epochs, Adam with $\text{lr}=2\text{e-}4$, $\beta_1 = 0.5$, $\beta_2 = 0.999$).

Why removing BatchNorm exacerbates mode collapse.

BatchNorm normalises each feature map across the batch:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \cdot \gamma + \beta.$$

This has two effects critical to GANs:

1. **Prevents internal covariate shift.** Without BN, the activation distribution at each layer shifts as the weights update. Deep generators can saturate or collapse to a constant output, mapping all $z \sim p(z)$ through the same path and collapsing the output to a single mode.
2. **Enforces batch diversity.** BN normalises *across the batch dimension*. If all B samples in a batch are nearly identical (a symptom of mode collapse), then $\sigma_B^2 \approx 0$, creating a large, destabilising gradient signal that acts as a diversity-restoring force. Without this, the generator can freely collapse all outputs to a single image x^* that best fools the discriminator:

$$G_\theta(z) \approx x^* \quad \forall z \sim p(z).$$

The no-BN generator showed early instability (G loss spike to ≈ 10.2 at epoch 10) before partially recovering, which is a typical sign of training instability linked to poor gradient conditioning.

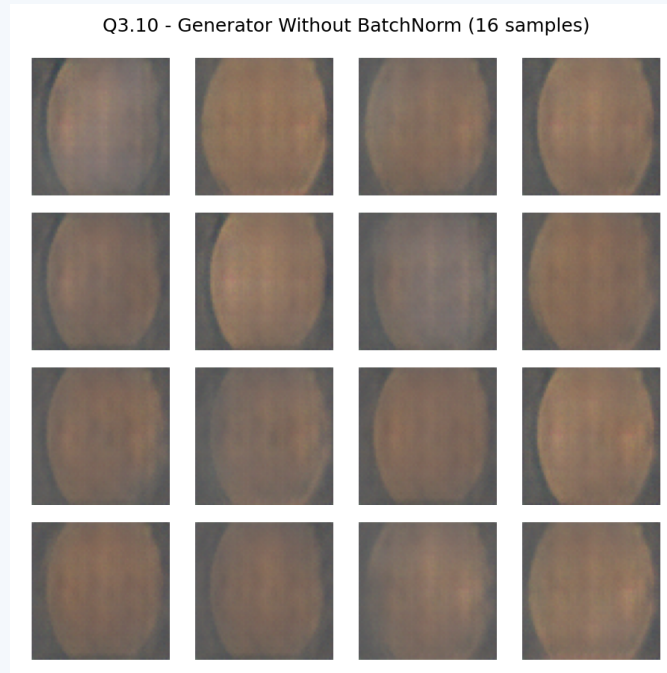


Figure 5: 16 samples from the Generator trained *without* BatchNorm on APTOS Class-4 images. Compared to the standard DCGAN, outputs show reduced diversity and more artefacts, consistent with the onset of mode collapse.

Phase 4: Self-Supervised Learning via SimCLR (PlantVillage)

4.11 InfoNCE Loss

SimCLR learns representations by contrasting two independently augmented views of each image. The augmentation pipeline applies random resized crops, horizontal flips, colour jitter, and random grayscale, which forces the encoder to learn invariance to these transformations.

NT-Xent (InfoNCE) Loss from scratch. Given a batch of N image pairs, the $2N$ projected embeddings $z_1, z_2 \in \mathbb{R}^{128}$ are L2-normalised and concatenated. The loss treats the corresponding pair (z_i, z_{i+N}) as a positive and all other $2N - 2$ entries as negatives:

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{2N} \sum_{i=1}^{2N} \log \frac{\exp(\text{sim}(z_i, z_{i+N})/\tau)}{\sum_{j \neq i} \exp(\text{sim}(z_i, z_j)/\tau)},$$

where $\text{sim}(u, v) = u^\top v / (\|u\| \|v\|)$ and $\tau = 0.5$ is the temperature. This is implemented as standard cross-entropy after masking the diagonal (self-similarity) with $-\infty$.

Model architecture. A ResNet-18 backbone (with the original classification head replaced by `nn.Identity()`) serves as the encoder, outputting 512-d representations. A two-layer MLP projection head maps these to a 128-d latent space for contrastive training. Labels are entirely ignored during pre-training.

Epoch	Train NT-Xent	Val NT-Xent
5	4.6684	—
10	4.5657	—
20	4.4906	—
30	4.4557	4.4566

The final NT-Xent loss on the validation set is **4.4566**. A lower NT-Xent corresponds to the encoder mapping the same image's two views closer together in embedding space relative to all other images in the batch.

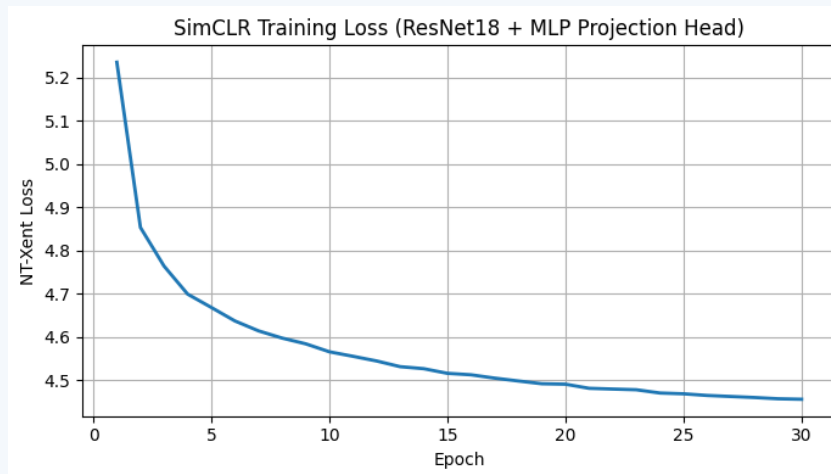


Figure 6: SimCLR contrastive training loss (NT-Xent) over 30 epochs on PlantVillage. The loss steadily decreases as the ResNet-18 encoder learns to align the two augmented views of each image.

4.12 The Linear Evaluation Protocol

To evaluate the quality of the learned representations without fine-tuning, the SimCLR encoder weights are fully frozen and a single untrained `nn.Linear(512, 38)` classification head is trained on top using true PlantVillage labels.

Why this is a fair evaluation. Freezing the encoder ensures that the linear classifier cannot reshape the feature space; it can only read off linearly separable structure already present in the 512-d representations. A high linear accuracy therefore implies that the contrastive pre-training has produced a well-structured, class-discriminative embedding space — without ever seeing class labels.

Model	Val Accuracy	Test Accuracy
SimCLR + Linear head (self-supervised)	92.68%	91.46%
Supervised CNN from Assignment 2	—	91.94%
Gap (Supervised – SimCLR)	0.48 pp	

The SimCLR linear evaluation achieves **91.46%** test accuracy — remarkably close to the **91.94%** of the fully supervised CNN from Assignment 2, with a gap of only 0.48 percentage points. This suggests that the ResNet-18 encoder, trained purely with contrastive loss, has learned representations almost as discriminative as those trained with direct label supervision. The result is notable because SimCLR uses a substantially larger backbone (ResNet-18 vs. a 3-block custom CNN), which contributes to its competitive performance.

Phase 5: Policy Gradient Theorem (Air Quality RL)

5.13 REINFORCE Implementation

MDP Formulation.

- **State:** s_t = past 24 hours of meteorological data, flattened to $\mathbb{R}^{168}$ ($= 24 \times 7$ features).

- **Action:** $a \in \{0, 1\}$ — 0 = purifier OFF, 1 = purifier ON.
- **Reward:** $R(s, a) = -0.1 \cdot \mathbf{1}[a = 1] - 10.0 \cdot \mathbf{1}[a = 0, \text{PM}_{2.5}^{t+1} > 150]$.

Policy Network. A single-hidden-layer MLP implemented in pure NumPy:

$$h = \tanh(W_1 s + b_1), \quad \pi_\theta(a = 1|s) = \sigma(W_2 h + b_2),$$

with $\dim(h) = 32$. The output is the probability of turning the purifier ON.

REINFORCE Algorithm. For each episode, a trajectory $\tau = \{(s_t, a_t, r_t)\}_{t=0}^{T-1}$ of length $T = 200$ is collected. The discounted return from step t is

$$G_t = \sum_{k \geq t} \gamma^{k-t} r_k, \quad \gamma = 0.99.$$

The policy gradient theorem gives the update direction:

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau \left[\sum_t G_t \nabla_\theta \log \pi_\theta(a_t|s_t) \right].$$

The gradient of $\log \pi_\theta$ is computed via the log-derivative trick:

$$\nabla_\theta \log \pi_\theta(a|s) = (a - \pi_\theta(a = 1|s)) \cdot \nabla_\theta \text{logit}(s).$$

A baseline $\bar{G}$ (mean return over the episode) is subtracted to reduce variance. The weight update is:

$$\theta \leftarrow \theta + \alpha \sum_t (G_t - \bar{G}) \nabla_\theta \log \pi_\theta(a_t|s_t).$$

The model is trained over 500 episodes, with each episode starting at a random position in the training time-series.

```
# Reverse-accumulated discounted returns
G = 0.0
for t in reversed(range(EP_LEN)):
    G = rewards_list[t] + GAMMA * G
    returns_list[t] = G
# Subtract baseline
G_mean = np.mean(returns_list)
returns_list = [G - G_mean for G in returns_list]
# Parameter update
for key in theta:
    total = sum(G * g[key] for G, g in zip(returns_list, grads_list))
    theta[key] += lr * total
```

Episode Range	Avg Return	Observation
1-50	-497.34	Random policy, high health penalties
100	-370.78	Policy starts learning to act on high PM days
300	-140.31	Significant improvement, fewer health penalties
500	-40.50	Policy has largely converged

The average return improves monotonically from around -500 to -40.5 over 500 episodes, which is a $\approx 12\times$ improvement. This shows that the REINFORCE algorithm has learned a reasonable strategy: the policy learns to turn on the purifier when meteorological conditions predict high $\text{PM}_{2.5}$, paying the small energy cost (-0.1) to avoid the large health penalty (-10.0).

The final 50-episode average return is **-40.50**.